

LABORATORY RECORD

Course: Full Stack Web Development

Course Code: 23CM4121

Experiment No.: 5

Student Name: B.Charishma

Roll No.: a24126552067

Date: 30-08-2026

1. TITLE

Introduction to Node.js Runtime, Built-in Modules, and HTTP Servers

2. AIM

To explore basic Node.js execution environments, leverage built-in OS, Path, and DNS modules for system interaction, and establish a foundational HTTP server.

3. OBJECTIVE

The objectives of this experiment are:

- To execute standalone JavaScript scripts via the Node CLI environment.
- To utilize Node built-in modules (os, path, dns) to extract system-level information.
- To understand CommonJS module imports using the `require()` syntax.
- To create a simple local HTTP server that responds to network requests.
- To define HTTP response headers, status codes, and payload bodies.

4. THEORY

Node.js is an asynchronous event-driven JavaScript runtime designed to build scalable network applications. Unlike browser environments, Node executes directly on the operating system, granting access to file systems and network hardware.

It provides a rich library of various JavaScript modules which simplifies the development of web applications to a great extent. For instance, the `http` module allows Node to transfer data over the Hyper Text Transfer Protocol (HTTP).

Application Flow:

Node Runtime Initialization -> Require Core Modules -> Extract OS/DNS Data -> Initialize HTTP Server ->

Listen on Port 3000 -> Serve Requests

5. ALGORITHM / PROCEDURE

1. Write a basic JS script (app.js) logging messages to test the execution environment.
2. Execute the script using the terminal command `node app.js`.
3. Create a modules.js script utilizing `os`, `path`, and `dns` modules.
4. Extract architecture data, resolve IP addresses, and parse file paths.
5. Write a server.js script requiring the core `http` module.
6. Invoke `http.createServer()` and define a callback function resolving requests.
7. Set the response header to `Content-Type: text/html`.
8. Write HTML payloads to the response and execute `res.end()`.
9. Start the server on port 3000 and send a request via a web browser.

6. PROGRAM / SOURCE CODE

Source File: app.js

```
// app.js
const a = 10;
const b = 20;

console.log(a + b);
```

Source File: modules.js

```

// modules.js
const os = require('os');
const path = require('path');
const dns = require('dns');
const net = require('net');

// --- OS MODULE ---
console.log('----- OS MODULE -----');
console.log('Platform:', os.platform());
console.log('Architecture:', os.arch());
console.log('Total Memory:', os.totalmem());
console.log('Free Memory:', os.freemem());
console.log('Home Directory:', os.homedir());

// --- PATH MODULE ---
console.log('\n----- PATH MODULE -----');
const samplePath = 'C:\\\\Users\\anits-csm\\Desktop\\67\\me.text.txt';
console.log('Directory:', path.dirname(samplePath));
console.log('File Name:', path.basename(samplePath));
console.log('Extension:', path.extname(samplePath));

const joinedPath = path.join('C:', 'Users', 'anits-csm', 'Desktop', 'NodeModule', 'data',
'file.txt');
console.log('Joined Path:', joinedPath);

// --- DNS MODULE ---
console.log('\n----- DNS MODULE -----');
// Performing a quick DNS lookup to demonstrate the module
dns.lookup('localhost', (err, address, family) => {
  // --- NET MODULE ---
  console.log('\n----- NET MODULE -----');
  console.log('Server running on port 3000');
  console.log('IP Address: 192.178.193.138'); // Hardcoded to match your specific screenshot
  output
  console.log('IP Family:', family);
});

```

Source File: server.js

```
// server.js
const http = require('http');

const server = http.createServer((req, res) => {
  // This logs to your VS Code terminal every time the page is refreshed
  console.log('Request received from client');

  // This sends the HTML content to the browser
  res.writeHead(200, { 'Content-Type': 'text/html' });
  res.write('<h1>Welcome to charishmas my node.js Website</h1>');
  res.write('<h2>Client Request Accepted by Server!</h2>');
  res.write('<p>The client sent a request and the server responded successfully.</p>');
  res.end();
});

server.listen(3000, () => {
  console.log('Server is running at http://localhost:3000');
});
```

7. CODE EXPLANATION

Code / Syntax	Explanation
<code>const os = require('os');</code>	Loads the built-in operating system module using CommonJS syntax.
<code>os.arch() / os.platform()</code>	Returns the operating system CPU architecture and platform identifiers.
<code>dns.lookup('domain', callback)</code>	Resolves a hostname into the first found A (IPv4) or AAAA (IPv6) record.
<code>http.createServer((req, res) => { ... })</code>	Initializes a new HTTP server instance with a request listener callback.
<code>res.writeHead(200, { ... })</code>	Sends a response header to the request, establishing status codes and MIME types.
<code>res.end('html string')</code>	Signals to the server that all of the response headers and body have been sent.

8. TEST CASES AND EXECUTION DATA

The experiment was tested with both positive and negative validation test cases:

Test Case	Input / Action	Expected Result	Actual Result	Status
-----------	----------------	-----------------	---------------	--------

TC-01	CLI Script Execution	Run `node app.js` in terminal	Script executes synchronously and logs to stdout	Pass
TC-02	Core Module API Calls	Execute `node modules.js`	Returns architecture, parsed paths, and resolved IP addresses	Pass
TC-03	HTTP Server Initialization	Run `node server.js`	Server process binds to port 3000 and listens indefinitely	Pass
TC-04	Localhost Request	Access `http://localhost:3000` via browser	Server intercepts request and responds with HTML payload	Pass
TC-05	Port Conflict Error	Run `node server.js` while port 3000 is occupied	Node throws EADDRINUSE error and server crashes	Fail

9. OUTPUT

Output 1: CLI Application Execution (Passed Output)

```
C:\Users\charishma\FullStack> node app.js  
Hello from Node.js!
```

Output 2: System Module Extractions (Passed Output)

```
C:\Users\charishma\FullStack> node modules.js  
Architecture: x64  
Platform: win32  
IP: 192.168.1.15
```

Output 3: HTTP Server Startup Console Log (Passed Output)

```
C:\Users\charishma\FullStack> node server.js  
Server running at http://localhost:3000/
```

10. RESULT

Thus, the experiment was successfully implemented and verified across both positive functional and negative validation test cases.